

初期化子リスト — 宣言と同時に値を入れる

今日のゴール

`int a[3] = {1, 2, 3};` と `char s[6] = "hello";` を、ローカル変数でもグローバル変数でも書けるようにする。

1 この回の位置づけ

- 言語機能発展シリーズ（選択制）。前提はコマ 14（グローバル変数）
- コマ 14 の資料が「配列や構造体の初期化リスト、文字列による初期化は扱わない」と明示的に先送りした宿題の回収
- `mycc.py` も `scaffold/parser.py` も書き換えない。ラッパー `langcc.py`（完成品）がパーサとコード生成の両方に差し込む
- 編集するのは `initializer.py` の 3 つの関数

2 いまはどうなっているか

```
int a[3] = {1, 2, 3};
```

いまのコンパイラに食わせると、**構文解析で止まる**。

構文解析エラー: [line 1] 式が期待されましたが '{' がありました

`=` の右側は式として読まれるが、C の式は `{` で始まらない。`{...}` は初期化子という、式とは別の構文である。

そのため、この回では 2 か所に手を入れる必要がある。

段階	追加すること
構文解析	<code>{ e1, e2, ... }</code> を読んで木にする
コード生成	各要素を、正しい場所書き込む

構文解析側はラッパーが用意する（F2～F4 で作ったパーサの拡張と同じ形で、`parse_expr` の先頭に「{」なら初期化子リスト」の分岐を足しているだけである）。できる木はこうなる。

```
kind='InitList', args=[Num(1), Num(2), Num(3)]
```

3 書かなかった要素は 0 になる

C には、初期化子について明確な規則がある。

```
int a[5] = {7, 8};    /* a[2], a[3], a[4] は 0 になる */
int b[3] = {};        /* 全部 0 */
```

足りない分は 0 で埋める。これは「たまたま 0 だった」ではなく、規格で決まった動作である（コマ 14 で見た `.bss` が 0 クリアされるのと同じ考え方）。

文字列による初期化も同じ枠組みで扱える。

```
char s[6] = "hello";  /* 'h','e','l','l','o','\0' */
char t[8] = "hi";     /* 'h','i','\0', 0,0,0,0,0 */
```

文字列リテラルは「文字コードの並び+NUL 終端」なので、`{104, 101, 108, 108, 111, 0}` と書いたのと同じことになる。

4 ローカルとグローバルで作りが違う

ここがこの回の面白いところである。同じ `= {1, 2, 3}` でも、置き場所によってまったく違うものが出力される。

	ローカル（スタック）	グローバル（ <code>.data</code> ）
いつ値が入るか	実行時（関数に入るたび）	実行前（プログラムの一部として）
出力するもの	命令（ <code>li</code> と <code>sw</code> ）	データ（ <code>.word</code> 列）

```
;; ローカル: int a[3] = {1,2,3};
li a0, 1
addi a1, s0, -24
sw a0, 0(a1)
li a0, 2
addi a1, s0, -20    ← 要素サイズぶん進む
sw a0, 0(a1)
...
```

```
;; グローバル: int g[4] = {10,20,30,40};
.data
.globl g
g:
.word 10
.word 20
.word 30
.word 40
```

グローバルのほうは**命令が1つも要らない**。実行ファイルの中に値がそのまま埋め込まれ、OS がメモリに読み込んだ時点で初期化が終わっている。コマ 14 の `.data / .bss` の話が、ここで効いてくる。

補足

だから「グローバルの初期値は定数だけ」なのか。コマ 14 で「グローバル変数の初期値は整数定数だけに限定する」と書いたのは、実行前に値が決まっていなければ `.word` に書けないからである。`int g = f();` が書けないのは、コンパイラの手抜きではなく、そもそも実行前に値が存在しないからである。

5 実装

Step	実装対象	内容
1	<code>init_values</code>	初期化子 → 値のリスト (0 埋め・文字列展開)
2	<code>gen_local_init</code>	命令で 1 要素ずつ書き込む
3	<code>global_init_data</code>	<code>.word / .byte</code> の行を並べる

Step 1 が共通の土台で、Step 2・3 はその結果を「命令にするか、データにするか」の違いだけである。

```
python3 langcc.py tests/global_array.c | head -12    # .data を見る
python3 langcc.py tests/local_array.c | grep -A2 "li a0, 1"
```

6 テスト

```
python3 check.py      # Step ごとの単体テスト（未実装は SKIP）
python3 golden.py     # 初期化子のテスト + fixed15
```

テスト	確認内容
local_array.c	ローカル配列の初期化
partial.c	足りない要素が 0 になる
global_array.c	グローバル配列が .data に置かれる
string_init.c	char s[6] = "hello" と NUL 終端

7 発展課題

1. サイズの省略: `int a[] = {1, 2, 3};` は要素数を初期化子から決める。配列の大きさが決まるのはパーサか、コード生成か。どちらで直すべきか
2. 構造体の初期化: `Point p = {3, 4};` を追加する。配列と違い、要素ごとにサイズもオフセットも違う（コマ 12 の `_struct_defs` を使う）
3. 入れ子の初期化子: `int m[2][2] = {{1, 2}, {3, 4}};`（多次元配列は Core プロファイル外なので、まず型の表現から考える必要がある）
4. 文字列リテラルとの違い: `char s[] = "hi";` と `char *p = "hi";` はまったく違うものである。前者はコピー、後者は `.data` への参照。生成コードを見比べて確かめる
5. B1 と組み合わせる: `int a[3] = {1+1, 2*3, 10/2};` は動くか。定数畳み込み（B1）があれば `.word 2 / .word 6 / .word 5` にできる

補足

コラム: 初期化と代入は違う。 `int a[3] = {1,2,3};` は初期化、 `a = {1,2,3};` は (C では) 書けない。配列には代入できないからである。一方 `int x = 5;` と `int x; x = 5;` は、C ではほぼ同じ意味になる (C++ では意味が違う場面がある)。「宣言と同時に値を与える」ことを別扱いにするのは、まだ何も入っていない場所に最初の値を置くという、代入とは別の操作だからである。